AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory

Hongju Kal
Yonsei University
Seoul, Korea
hongju.kal@yonsei.ac.kr

Chanyoung Yoo
Yonsei University
Seoul, Korea
chanyoung.yoo@yonsei.ac.kr

Won Woo Ro
Yonsei University
Seoul, Korea
wro@yonsei.ac.kr

ABSTRACT

This paper presents an asynchronous execution scheme to leverage the bank-level parallelism of near-bank processing-in-memory (PIM). We observe that performing memory operations underutilizes the parallelism of PIM computation because near-bank PIMs are designated to operate all banks synchronously. The all-bank computation can be delayed when one of the banks performs the basic memory commands, such as read/write requests and activation/precharge operations. We aim to mitigate the throughput degradation and especially focus on execution delay caused by activation/precharge operations. For all-bank execution accessing the same row of all banks, a large number of activation/precharge operations inevitably occur. Considering the timing parameter limiting the rate of row-open operations (tFAW), the throughput might decrease even further. To resolve this activation/precharge overhead, we propose AESPA, a new parallel execution scheme that operates banks asynchronously. AESPA is different from the previous synchronous execution in that (1) the compute command of AESPA targets a single bank, and (2) each processing unit computes data stored in multiple DRAM columns. By doing so, while one bank computes multiple DRAM columns, the memory controller issues activation/precharge or PIM compute commands to other banks. Thus, AESPA hides the activation latency of PIM computation and fully utilizes the aggregated bandwidth of the banks. For this, we modify hardware and software to support vector and matrix computation of previous near-bank PIM architectures. In particular, we change the matrix-vector multiplication based on an inner product to fit it on AESPA PIM. Previous matrix-vector multiplication requires data broadcasting and simultaneous computation across all processing units. By changing the matrix-vector multiplication method, AESPA PIM can transfer data to respective processing units and start computation asynchronously. As a result, the near-bank PIMs adopting AESPA achieve 33.5% and 59.5% speedup compared to two different state-of-the-art PIMs.

CCS CONCEPTS

• **Computer systems organization → Parallel architectures**.

KEYWORDS

Processing-in-Memory, Execution Method, Matrix-Vector Multiplication

ACM Reference Format:
Hongju Kal, Chanyoung Yoo, and Won Woo Ro. 2023. AESPA: Asynchronous Execution Scheme to Exploit Bank-Level Parallelism of Processing-in-Memory. In *56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23), October 28–November 01, 2023, Toronto, ON, Canada.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3613424.3614314

MICRO '23, October 28–November 01, 2023, Toronto, ON, Canada

ACM ISBN 979-8-4007-0329-4/23/10…$15.00
https://doi.org/10.1145/3613424.3614314

1 INTRODUCTION

Near-bank processing-in-memory (PIM) is specialized to accelerate memory-intensive workloads by utilizing the aggregated bandwidth of multiple DRAM banks [6, 12, 28, 36]. The near-bank PIM has processing units assigned to banks, which simultaneously compute data from the dedicated bank. Such an approach leads to an increase in data bandwidth and computational throughput in proportion to the large number of banks. By leveraging the high bandwidth, the near-bank PIM mainly targets vector and matrix operations [10–12, 28] that are basic building blocks of neural network (NN) applications [8, 9, 13, 24, 37, 40].

For the parallel execution of processing units, previous near-bank PIM architectures, PIM-HBM [22, 23, 28] and Newton [12, 26, 29], have adopted a single-instruction multiple-data (SIMD) execution. These architectures send one command at a time through the channel (single instruction), and then the processing units compute data stored in local banks (multiple data). Contrary to DRAM architecture which is supposed to access a single bank asynchronously, they are designed to access all banks synchronously. To operate processing units of all banks, they adopt a new command broadcasted across the banks, called all-bank command. By using the all-bank command, all processing units retrieve data from local banks placed in the same row/column addresses. The data retrieved from the bank is computed with local data stored in their SRAM structure or global data transferred from the channel bus.

However, such an SIMD execution model suffers from low bank-level parallelism, especially when conventional DRAM commands are interposed into a stream of all-bank commands. The conventional DRAM commands are usually per-bank commands – such as read/write requests from the host processor, activation/precharge, and per-bank refresh [3] – which cannot be performed with all-bank commands together. If one bank is performing a per-bank command, the memory controller cannot issue the all-bank command because of the bank performing the per-bank command. The computation of the all-bank command is delayed while the single bank performs the per-bank command even though the rest banks are idle.

In this paper, we mainly focus on activation/precharge commands because the PIMs inevitably require frequent operations of these commands for data access, since the other conventional commands rarely or do not occur by limiting the memory to perform PIM operations only [4, 12, 28]. During all-bank execution, the near-bank PIM requires a number of activation/precharge commands to open and close all rows of the numerous banks. While one bank performs activation/precharge, the memory controller cannot issue the all-bank command. This overhead is significant because the latency of the activation/precharge command (tens of nanoseconds) is longer than that of the all-bank command (a few nanoseconds) [38, 39]. Furthermore, in the case of the activation commands, the number of banks that can be activated is limited within a time window, such as tFAW. Therefore, the latency of activation is considerable, which takes 43-70% of the total execution time in our initial evaluation.

We propose **AESPA**, an **A**synchronous **E**xecution **S**cheme exploiting bank-level **PA**rallelism, to address the above issue. Since such inefficient execution is caused by the difference in command type (i.e., per-bank and all-bank), and therefore AESPA adopts a new per-bank command, which we name as long-data command. Contrary to all-bank and conventional read/write commands, this command makes a processing unit compute an entire DRAM row of the target bank. While the processing unit is computing the data, the channel can send additional long-data commands or other per-bank commands to other banks. The execution time of the long-data command is long enough to issue multiple long-data commands across all other banks and make their processing unit start execution. By doing so, AESPA asynchronously starts execution but operates all banks concurrently, thereby preserving high bank-level parallelism. We use the term single-instruction long-data (SILD) execution, which is distinct from the SIMD execution. It represents that this mechanism uses one command (single instruction) to compute data of the entire row (long data) instead of data of all banks (multiple data).

AESPA aims to support element-wise vector-vector computation and matrix-vector multiplication, all operations available in the previous PIM architectures. The element-wise vector-vector computation is easily supported because each processing unit locally computes its dedicated data without shared data. Thus, the processing units asynchronously start their dedicated computation by adopting long-data commands only. On the other hand, hardware/software modifications are required to handle matrix-vector multiplication. In previous PIM architectures, all-bank commands broadcast vector data across all processing units via channel bus for matrix-vector multiplication. The broadcast transfer cannot be performed by the long-data command since it is based on the per-bank command.

Therefore, this paper explores redesigning the computing method and hardware to enable matrix-vector multiplication on AESPA PIM. For this, we change the computing method of matrix-vector multiplication and the configuration of processing units for the changed matrix-vector multiplication. Instead of currently-used matrix-vector multiplication that consists of inner products of one vector and matrix rows (element-wise vector-vector multiplications and vector accumulations), AESPA PIM conducts multiplications of vector elements and matrix columns (scalar-vector multiplications) and summations of the multiplication results (element-wise vector-vector summations). The multiplications are performed by multipliers of processing units by using vector data received from the channel bus and matrix column data stored in the local banks. For the summations, AESPA PIM places adders in bank groups and uses them as shared resources across banks, which are placed in the processing units and locally compute data in the previous architectures.

As a result, AESPA PIM reduces all bank activations overhead and further hides the latency of the single activation. We adopt the AESPA PIM design to two different types of memory devices, HBM2 and GDDR6, which are used for PIM-HBM and Newton, respectively. Since PIM-HBM and Newton have different execution mechanisms, we provide two different PIM hardware types similar to these architectures, respectively. The AESPA-based PIM achieves 33.5% speedup on HBM2 memory configuration compared to PIM-HBM, and 59.5% speedup on GDDR6 memory configuration compared to Newton.

This paper makes the following contributions.

- We provide AESPA which uses the new parallel execution method, SILD that is distinct from SIMD. AEPSA not only maintains asynchronous behaviors of the memory but also exploits bank-level parallelism of the near-bank PIM.
- We present AESPA PIM design in which data placement and processing core are redesigned to support AESPA. The hardware modification for AESPA is feasible in terms of area and power overhead. Also, this modification is easily adapted to previous architectures.
- We evaluate AESPA PIM in terms of parallelism and computational throughput compared to the previous PIM architectures. AESPA outperforms previous all-bank execution scheme with various memory organizations.

2 NEAR-BANK PIM DESIGN

This section introduces general matrix-vector multiplication (GEMV) commonly used in conventional computing and the data placement of the GEMV in near-bank PIM designs. Also, we explore the all-bank execution mechanism of near-bank PIM. Our study explores two near-bank PIMs - Samsung PIM-HBM [23] and SK Hynix Newton [29] - that are real-word PIM adopting SIMD execution. Based on the near-bank PIM designs described in this section, we will explain how to resolve the bottleneck of all-bank execution and reconstruct the near-bank PIM design in Section 3 and 4.

2.1 GEMV in Near-Bank PIM

GEMV is a basic building block of various applications, such as neural networks and graph analytics [31, 35]. It is a memory-bound operation requiring massive bandwidth in that the large matrix data is received from the off-chip memory but is used once. Since the low data reuse rate of the matrix, on-chip memory cannot be utilized and the performance of GEMV depends on the off-chip bandwidth. Therefore, the near-bank PIM has become a promising solution for GEMV in that it leverages the internal bandwidth of all banks [6, 12, 28].

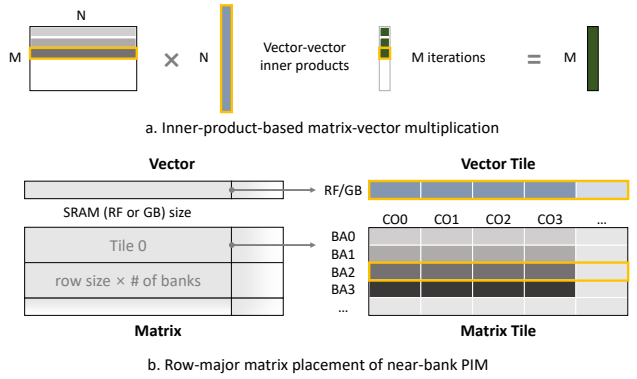

Figure 1: Inner-product-based GEMV and its data placement in near-bank PIMs

The GEMV operation used for PIM is based on the inner product, widely used in various linear algebra routines (BLAS) [2, 14]. Figure 1.a illustrates that GEMV involves multiple inner products of the matrix row and vector. The inner products are performed by a series of multiply-and-accumulate (MAC) operations. The MAC operation calculates each matrix row and the shared vector, which is repeated for each matrix row with the same pattern. Therefore, this computing method is suitable for executing in SIMD processors with high thread-level parallelism.

Figure 1.b presents the matrix and vector data placement to show how the near-bank PIMs exploit bank-level parallelism. To allocate data, the vector and matrix must be divided into tiles, the basic blocks of parallel execution. In fact, the size of vector and matrix tiles depends on hardware configurations, such as the number of banks and the size of the internal SRAM structure. Since the vector tile is input data and stored in the SRAM (the register file of PIM-HBM and the global buffer of Newton), the size of the vector tile is normally determined by the size of the SRAM storage. On the other hand, the matrix tile is stored in the DRAM row of all banks, and its size is equal to the size of the DRAM row multiplied by the number of banks. The SRAM buffer and DRAM columns provide a 32-byte vector and the same-sized matrix data to the 16 multipliers so that sixteen pairs of 2-byte floating-point values are multiplied simultaneously. After data allocation, the memory controller broadcasts a command across banks to perform GEMV. Then, each processing unit computes the vector from the SRAM entry and the matrix row from the DRAM column in all banks in a lockstep manner.

2.2 Near-Bank PIM Architecture

Near-bank PIMs that place processing units for banks have been proposed to accelerate data-intensive applications. Each processing unit can simultaneously retrieve data from its own bank, utilizing bank-level parallelism. The near-bank PIM can achieve higher bandwidth in proportion to the number of banks than other memory-centric accelerators that exploit rank- or channel-level parallelism [17, 18, 20, 25]. In order to utilize the bandwidth of all banks simultaneously, an all-bank execution mechanism is essential. Since the host processor controls DRAMs through conventional DRAM commands, the PIM also uses DRAM commands that initiate simultaneous operations and broadcast data to multiple banks. In the near-bank PIM architecture, multiple banks can transfer data from the DRAM column to the dedicated processing unit. The all-bank execution is available without data congestion since the banks use their local data bus only for concurrent data accesses.

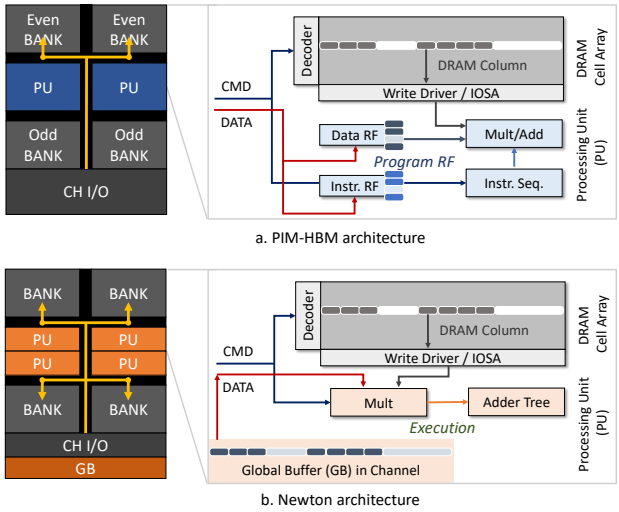

Figure 2: Basic operations of the state-of-the-art near-bank PIM architectures. (a) PIM-HBM architecture [23] and (b) Newton architecture [29].

Figure 2 shows the PIM organization of PIM-HBM and Newton. We consider only four banks and the basic per-bank operations for a fair comparison regardless of DRAM types (HBM2 and GDDR6). As shown in Figure 2.a, PIM-HBM adopts a processing unit for a pair of even and odd banks (e.g., bank 0 and bank 1). Because even and odd banks share the processing unit, the processing unit computes data from only one bank in a single DRAM column access. Thus, the single DRAM command enables half of the banks to retrieve data, called all-bank mode. The right side of the figure illustrates how the processing unit performs PIM computation. The processing unit has a SIMD multiplier and adder for a pair of banks and registers files assigned to even and odd banks, respectively. Before the PIM operation, when programming the register file, the host processor sends the same sequence of instructions and data to all processing units via all-bank commands. Then, when a bank receives an all-bank command, the command causes the instructions in the register file to be issued to the processing unit. All processing units receive data from banks and register files and perform the same instruction in a lock-step manner.

Newton has a dedicated processing unit per bank that employs SIMD multiplier and adder tree specialized for multiply and accumulate (MAC) operation, as shown in Figure 2.b. Contrary to PIM-HBM exploiting half the parallelism of all banks, the command is broadcasted to all processing units and has parallelism for all banks. Also, it adopts the global buffer in the bank group that stores data required by all processing units instead of separated register

files in the banks. While PIM-HBM uses commands to issue instructions and retrieve data stored in register files, Newton directly uses the commands and the data transferred from the channel. During the PIM operation, the data of the global buffer is transferred to the processing units through the global data bus by the all-bank command. By doing so, all processing units perform MAC operations concurrently within a single-column access delay.

3 RECONSTRUCTING PIM EXECUTION

We address the inherent problem of all-bank execution and present AESPA as a solution. This section introduces the basic concept of the proposed execution model, and the detailed implementation of our design is covered in the following section.

3.1 Challenge of Execution

To perform PIM operations on near-bank PIM architectures, all banks should be activated before PIM execution. This is because the processing units use DRAM columns that exist in the same DRAM row as the sources of the operations. Therefore, the rows of all banks must be opened to perform PIM operations. For this, the memory channel should issue multiple activation commands sequentially as many times as the number of banks. However, after performing activation four times, the memory channel has to be stalled until the cycle reaches to the four-activation timing window (tFAW), that is prescribed by JEDEC standards as a timing constraint of memory devices for power concern [15, 16].

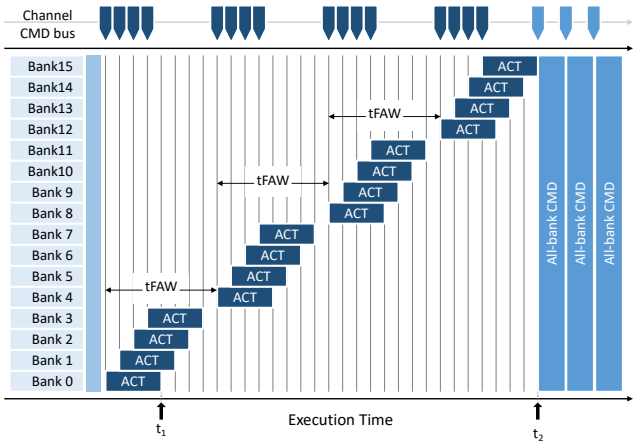

Figure 3: Concept of all-bank execution scheme. The service timeline is drawn not to scale. tACT and tFAW are much longer and no other timing parameters are presented.

To enable all bank commands simultaneously, all banks need to be stalled until the activation of each and every bank in the channel is completed. As shown in Figure 3, despite the completion of Bank 0 activation at time t_1, making it ready for execution by subsequent commands, the operation of Bank 0 is delayed until t_2. The total waiting time to activate N banks (i.e., 0 to t_2 in the figure) can be represented as tACTN in Equation (1). Since four activations are only allowed during tFAW, 4 × (N/4-1) banks are activated in tFAW × (N/4-1). After that, the rest banks as (N-1) mod 4 are activated sequentially in tACT + (N-1) mod 4 where tACT is the latency of a single bank activation. As other timing constraints to be considered, row-to-row delay (tRRD) exists, which is the minimum time interval between successive activations to different banks. The four activations cause three tRRD but this delay does not affect the activation time of Equation (1) because the tFAW timing window exceeds and hides three tRRDs.

$$tACTN = \left(\left\lceil \frac{N}{4} \right\rceil - 1\right) \times tFAW + tACT + (N-1) mod 4 \quad (1)$$

In near-bank PIM architectures, this latency overhead of activation is a major cause of overall execution delay. Figure 4 illustrates the processing time breakdown according to the command types in the near-bank PIM devices when performing a GEMV operation with various matrix sizes. In the case of the Newton architecture, PIM operations are performed using the COMP command, which is broadcast to all banks. The time required for the operation is only a small fraction of the total execution time by 28.62% whereas the time required for activation/precharge is as high as 70.33% of the total execution time. In the PIM-HBM architecture, PIM operations are performed using the READ command, which is sent to the device in all-bank mode. The time required for the operation is only 34.03% of the total processing time. Although this architecture has additional operations, such as writing input to registers or changing modes through the write commands, activation/precharge still takes 42.64%

Interestingly, the latency overhead of activations depends on the memory types and PIM architectures regardless of the matrix size. The activation delay depends on the number of banks and timing parameters as described in Equation (1). Meanwhile, the execution delay to compute the DRAM row is always equal to the sequential access time of entire columns. Considering that these delays are repeated whenever new DRAM rows are activated, the ratio of activation delay to execution time remains constant.

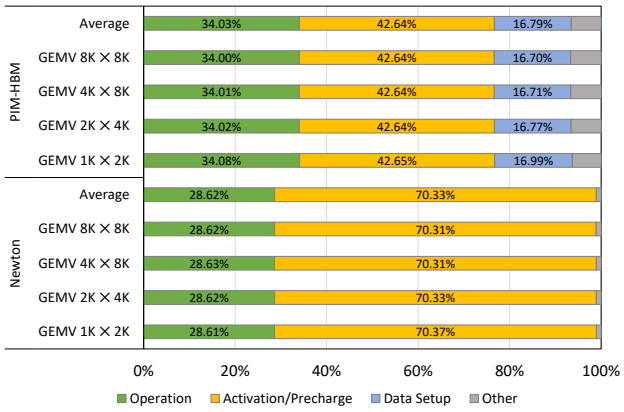

Figure 4: Distribution of processing time according to command types in near-bank PIM architecture

The all-bank activation not only increases the execution time of operations but also prevents achieving the expected throughput that can be obtained by multiple-bank parallelism. Low throughput

is because activations are serially executed and cause the sequential bottleneck (i.e., Amdahl's law). For example, a PIM architecture with N banks operating simultaneously is expected to achieve N times speedup but the delay caused by tACTN of Eq (1) is added, resulting in execution time delay compared to the ideal case.

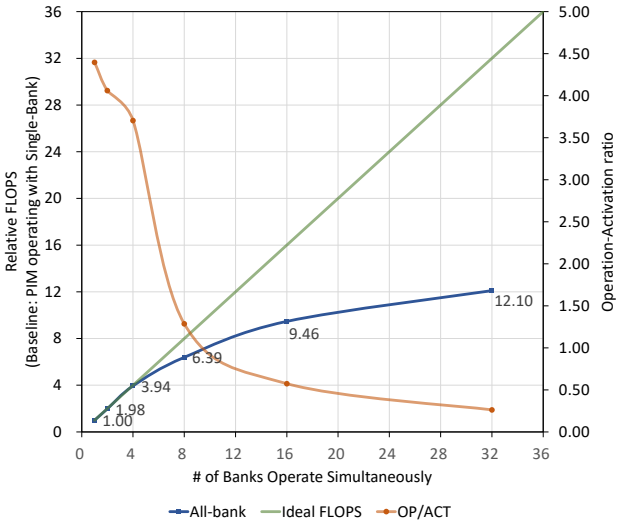

Figure 5: FLOPS increase according to the number of banks that can be operated simultaneously with a single command

Figure 5 shows the difference between the ideal case and the all-bank execution in terms of FLOPS by varying the bank-level parallelism. In our experiment, we use a single channel having 32 banks and vary the number of banks operating simultaneously with a single command. In an ideal case that has no activation latency, the throughput might be increased proportionally as the number of banks operating simultaneously increases. However, in the actual near-bank PIM architecture, if the number of banks operating simultaneously exceeds four, FLOPS no longer increases linearly. With 8 banks, FLOPS increases only 6.39×, with 12 banks it increases by only 9.46×. In the end, with 32 banks, the FLOPS increases 12.10×, which is only 37% of the ideal FLOPS. Such a low throughput increase comes from the activation overhead. It is demonstrated by the significant decrease in the ratio of activation delay to execution delay (as represented by the OP/ACT line in the graph). In other words, the negative impact caused by activation overhead is greater than the positive impact of parallelism. Therefore, it is required to alleviate activation overhead for exploiting the aggregated bandwidth of massive banks.

3.2 Solution of Execution: AESPA

The prior near-bank PIM architecture suffers from activation overhead that cannot be hidden by the execution time of other commands. To address this issue, we propose AESPA, an **A**synchronous **E**xecution **S**cheme for matrix multiplication exploiting bank-level **PA**rallelism. AESPA aims to hide activation delay while preserving the high bank-level parallelism. Thus, the underlying principles of the AESPA design are as follows: **(i) it operates asynchronously for different commands**, and **(ii) the processing unit computes its own data for a long cycle, enabling computation overlap of other processing units.**

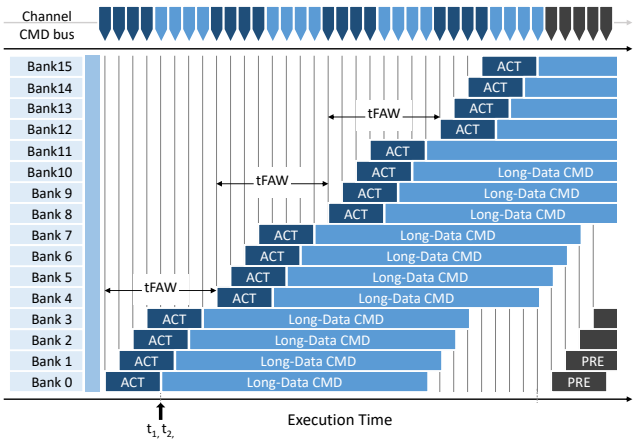

Figure 6: Concept of AESPA. The service timeline is drawn not to scale. tACT and tFAW are much longer and no other timing parameters are presented.

To satisfy **(i)** and **(ii)**, AESPA presents newly developed *long-data commands*; the long-data commands compute data of the entire row in the target bank different from the all-bank commands. Figure 6 illustrates how the timing delay changes when AESPA with the long-data commands is applied. Since each bank operates by an individual command, it can perform PIM operations as soon as its activation is completed regardless of the other banks' status. The previous execution scheme suffers from a stalled delay due to all-bank activation when the next compute command requires a row open (i.e., t_1 to t_2 in Figure 3). On the other hand, it is possible to hide the activation delay and continue the next operation immediately in AESPA. We name this novel execution concept as SILD (single-instruction long-data) execution to distinguish it from SIMD execution used in previous PIMs.

Table 1: Comparison of All-Bank and Long-Data Commands

Features	All-bank CMD	Long-data CMD
# of target	# of BA	# of CO
Issuing delay	tCCD	tCCD
Computing delay	tCCD	tCCD × # of CO
Concurrent CMDs	1	# of CO
Parallelism factor	# of BA	# of CO

Table 1 shows a comparison of the characteristics between the long-data command used in AESPA and the existing all-bank command. As mentioned earlier, the all-bank command targets one column of all banks, while the long-data command targets all opened columns (that is the entire row) of one bank. The memory controller can issue a long-data command to the single bank within a column-to-column delay (tCCD) likewise all-bank command. Meanwhile,

the computing time of the long-data command is the same as the delay of the entire column access (tCCD × # of CO). Therefore, while one bank receives the command and computes the entire columns, AESPA can concurrently operate a number of the commands equivalent to the number of DRAM columns. As a result, the all-bank command and long-data command can exploit parallelism as much as the number of banks and columns, respectively. However, if the number of DRAM columns exceeds the number of DRAM banks, the long-data command also exploits parallelism limited to the number of processing units (i.e., # of banks). Conversely, if the number of columns is lower than that of banks, the parallelism is restricted and equal to the number of columns. In fact, because most memory devices have more columns of a row than banks of a channel, the long-data commands can achieve the same level of parallelism as the all-bank command.

Figure 7 shows the delay breakdown when adopting AESPA to the near-bank PIM architectures. We set the baseline architectures to use all-bank commands and AESPA to use the long-data commands and show evaluation results when performing 2K×4K GEMV. The activation/precharge delay and operation delay in the figure demonstrate that AESPA successfully hides the activation/precharge latency. PIM-HBM shows that activation/precharge of total execution time decreases from 42.64% to 23.16%. In Newton, activation/precharge accounted for 70.33% of the total execution time while decreasing to 27.01% by adopting AESPA. Meanwhile, both PIM-HBM and Newton architectures show similar operation delays and the difference between the two baseline architectures and AESPA is negligible. It means that AESPA achieves the same level of parallelism as the previous scheme. When adopting AESPA, the latency of activation/precharge can be hidden while preserving the bank-level parallelism.

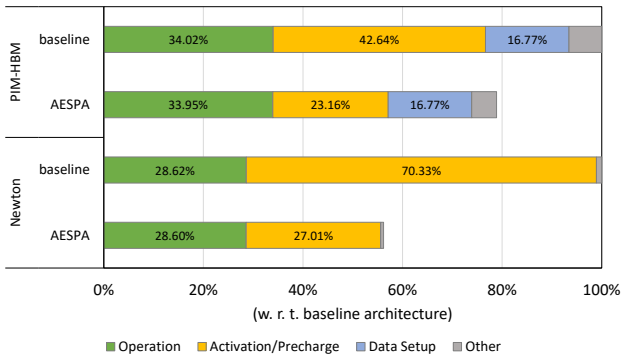

Figure 7: Distribution of processing time according to command types in AESPA PIMs

4 AESPA-BASED PIM DESIGN

In this section, we introduce PIMs that are modified to support the new execution scheme, AESPA. To utilize the long-data command instead of the all-bank command, it is essential to change the PIM design that includes data placement and processing units. The main reason is that vector data should only be sent to one bank at once by the long-data command. Then, the processing unit has to reuse the received vector data and compute it with the matrix data stored in the DRAM row. Therefore, high re-usability of the vector data is essential for GEMV operations and this section develops a new approach to achieve a high reuse rate. This entails a modification of the data placement and processing unit, which is explained in Section 4.1 and 4.2, respectively.

4.1 GEMV in AESPA PIM

The naive approach of matrix multiplications using the inner product cannot be applied for the proposed AESPA, the asynchronous execution. In the all-bank PIM designs, the vector broadcasted from a channel should be computed with matrix rows stored in each column of banks concurrently. If processing units compute their own multiplication of vector and matrix rows asynchronously, the different vectors are required by each of the processing units and they should be transferred at the same time. This is not possible since memory controller cannot send multiple data to multiple banks concurrently.

To overcome the above limitation, we change the computation order of GEMV and rearrange its data placement in our proposed architecture. The proposed AESPA sends dedicated vector data to the corresponding bank and makes their processing units reuse the received vector data repeatedly. To do so, the matrix is computed as column-major, different from the PIM-HBM and Newton methods where a sequence of blocks for a matrix row and the same vector are multiplied across all banks. The column-major GEMV is interpreted as Equation (2). We change GEMV to multiply matrix columns ($\overrightarrow{W}^{(n)}$) with vector elements (x_n) and accumulate them ($x_n\overrightarrow{W}^{(n)}$). For multiplication, AESPA PIM transfers a matrix data block to each bank via a long-data command so that its processing unit performs the multiplication of matrix column and vector element. On the other hand, the accumulation is performed across processing units, and thus hardware modification is required to gather multiplication results and perform accumulation.

$$\overrightarrow{y} = \overleftrightarrow{W}\overrightarrow{x} = x_1\overrightarrow{W}^{(1)} + x_2\overrightarrow{W}^{(2)} + \cdots + x_N\overrightarrow{W}^{(N)} \quad (2)$$

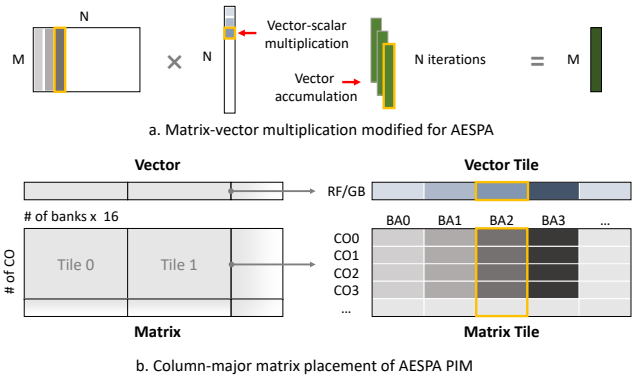

Figure 8: Proposed GEMV and its data placement in AESPA PIM

Figure 8.a shows the GEMV used in AESPA, which consists of vector-scalar multiplication and vector accumulation. The columns

of the matrix and the elements of the vector are multiplied in order and the results are accumulated in a single result vector. By doing so, a single vector element is reused for multiplying with the column elements of the matrix while preserving parallelism in multiplication. To perform the GEMV in AESPA PIM, we design the data placement that fits in the memory organization.

To reuse the data received by the long-data command, the data placement of the matrix needs to be changed. Figure 8.b represents the data placement of the matrix used in the AESPA PIM. First, tiling is required to place data before PIM operation, which is similar to the previous PIMs but we use a different tile size. To distribute the vector tiles across banks, the vector tile is composed of 32-byte data blocks (DRAM column size) as many as the number of banks. As the 32-byte data block stores sixteen 2-byte floating-point values, the size of the vector tile is equal to the number of banks multiplied by 16 (# of BA×16 in the figure). For the matrix tile, the width of the matrix tiles should be equal to the size of the vector tile since they are multiplied. The length of the matrix tile is equal to the number of DRAM columns computed by the single long-data command (# of CO in the figure). Therefore, 32-byte data blocks of the column direction are continuously stored in DRAM columns in the same DRAM bank (e.g., CO0, CO1, CO2, CO3, ... in BA2). After tiling, vector data needs to be multiplied with matrix data in the column direction, not the row direction.

4.2 AESPA PIM Architecture

AESPA PIM can be designed to resemble both styles of the PIM-HBM and Newton; PIM-HBM supports diverse operations by using programmable register file and Newton is a lightweight architecture specialized in GEMV only. We present two possible PIM designs based on PIM-HBM and Newton, as illustrated in Figure 9.a and Figure 9.b, respectively. For a fair comparison of AESPA and the all-bank execution scheme, the AESPA PIMs are implemented as a similar PIM core design with the same memory configuration. The main difference between the AESPA PIMs and the previous PIMs is the accumulator design. Previous designs adopt an accumulator per bank whereas AESPA PIMs place a shared accumulator in a bank group. During execution, the processing units of the bank group multiply the vector received from the channel and matrix row stored in the DRAM column. The result of multiplication in the bank group is saved in a shared accumulator and then can be accumulated.

The common computation procedure of AESPA PIMs (PIM-HBM- and Newton-based) is as follows; Firstly, a memory controller sends the long-data commands to each bank. The vector data is transferred with the commands in the Newton-based AEPSA PIM and already stored in the register file beforehand in the PIM-HBM-based AESPA PIM. The processing units compute multiplications of the received vector data and the matrix data from their banks and send the intermediate results to the accumulator. Then, the accumulator adds the partial sum if the intermediate results are the result of the same matrix row. This operation occurs in every column-to-column delay (tCCD) until all matrix rows are computed with the vector.

Most of the compute units are not different from those in the previous PIMs, but the accumulator needs to be newly designed for the accumulator should be able to sum the partial sums that

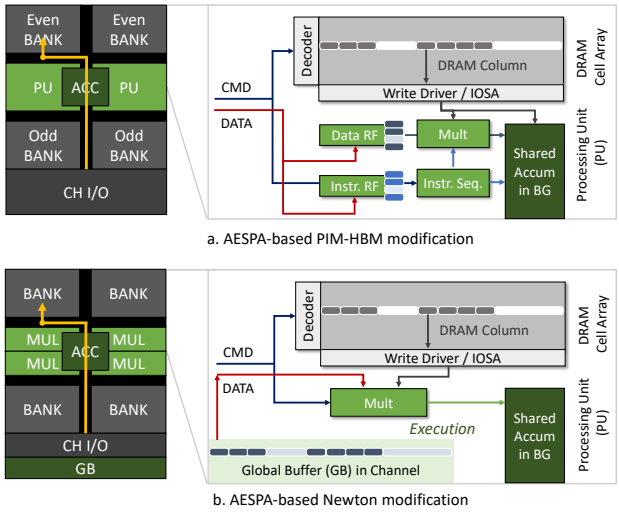

Figure 9: Basic operations of AESPA PIM architectures

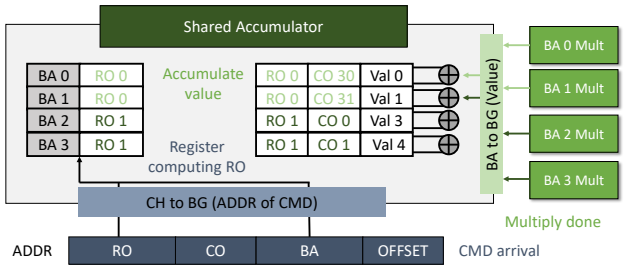

Figure 10: Accumulator design of AESPA PIM

are the results of multipliers across banks. The partial sums are asynchronously transferred to the accumulator, so the accumulator sums the intermediate values after identifying the matrix data location. To do so, the accumulator is designed, as shown in Figure 10. If the received results are not a part of the partial sum for the same matrix rows, the results should not be accumulated with each other. For identifying which multiplication results are accumulated, each long-data command registers the row address of the banks currently being computed. Then, the multiplication results, computed by the long-data command, are delivered in DRAM column order. The accumulator checks that the received values come from different processing units. Then, if they are multiplication results computing data of the same row and column addresses, the accumulator sums up the results.

5 DISCUSSIONS

Supporting various operations: In the near-bank PIM architectures, the PIM core is designed to accelerate only a few operations in an ASIC style, rather than supporting general-purpose processing. Thus, we mainly focus on GEMV which is a main target operation of various PIM designs. However, AESPA can be easily applied for

element-wise vector operations that are supported in the PIM-HBM design.

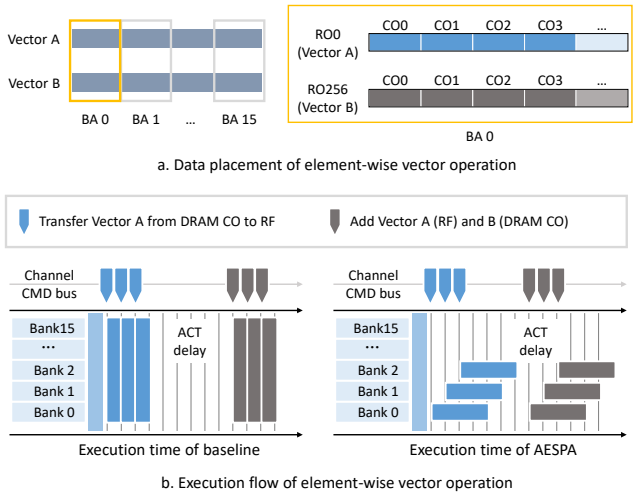

Figure 11: Element-wise operation in all-bank execution and AESPA

Figure 11 presents data placement and execution flow of element-wise vector operation in PIM-HBM when adopting all-bank execution or AESPA. Assume that PIM-HBM performs element-wise addition of Vector A and Vector B. In both schemes, Vector A and Vector B are evenly distributed and stored throughout the banks before execution begins. In the case of the all-bank execution, all processing units concurrently perform the same operation following the all-bank commands. First, all processing units transfer partial Vector A from the DRAM columns to the register file. After that, the processing unit computes the partial Vector A stored in the register file with partial vector B stored in other DRAM columns. These operations can be carried out asynchronously, as shown in the execution flow of AESPA. A long-data command makes one processing unit transfer Vector A of its own bank to the register file or add Vector A and Vector B. By issuing multiple long-data commands, AESPA can achieve a high throughput of element-wise vector operations as well. Contrary to GEMV requiring data broadcasted from the channel, the element-wise vector operations have no data transferred via channels. Therefore, they do not need to change the computing method and data placement of banks.

Other solution for activation overhead: In an effort to reduce the number of activation commands and minimize activation overhead, the Newton architecture employs ganged activation commands. Compared to the conventional activation command, this command targets four banks simultaneously and triggers their activation together (ACT4). However, this method still encounters limitations as it necessitates waiting for the tFAW, resulting in the same overhead issue. Thus, Newton proposes the incorporation of a voltage generator for DRAM cells, thereby reducing tFAW [12]. Also, GDDR6-AiM, which is the real-world PIM product of the Newton architecture, features a large reservoir capacitor for each bank, supplying additional charge [29]. The activation of GDDR6-AiM is not limited by tFAW and sixteen banks can be opened by a single activation command (ACT16). This solution is practical but causes the area overhead. In GDDR6-AiM, the area of the processing unit including the large capacitor is measured as 0.19 mm^2 per bank. Note that the area of the single bank is 0.687 mm^2 [43] used in GDDR6. Also, performing activations distributedly (per-bank activation) offers advantages in terms of achieving uniform power dissipation when compared to simultaneous activations across all banks (all-bank activation). Our study is worth proposing an architectural solution instead of the circuit-level approach. Moreover, AESPA not only prevents tFAW overhead but also hides the latency of a single activation. Therefore, it outperforms the ideal all-bank PIMs that synchronously operate all processing units without the tFAW constraint, which is shown in the later section.

Potential impact of AESPA: Our key observation is that the DRAM is supposed to operate asynchronously but PIM is designated to operate synchronously. Therefore, AESPA changes the PIM execution mechanism to operate asynchronously, thereby solving the mismatch of DRAM and PIM operations and reducing the execution delay. This applies not only to activation/precharge but also to other existing DRAM commands that are operated asynchronously, such as read/write commands requested from the host processor. In other words, the read/write commands can be performed with long-data commands of AESPA concurrently if their target banks are different. However, in the current memory system utilizing PIM, the read/write commands of the host process do not occur because the memory address space and physical memory device are separated for PIM and the host processor [6, 23, 29]. Such a memory physically separated for PIM and host memory has the disadvantage of dividing capacity and bandwidth, which leads to unbalanced bandwidth utilization and storage overhead of data replication. Thus, recent several studies [1, 4] have proposed to share the memory with the host processor and PIM. Once such a PIM system is developed, AESPA will have greater benefits in terms of fairness between host and PIM threads and memory performance.

Software modification for AESPA: Applying AESPA involves changing the execution scheme mainly and does not require significant modifications to the existing software stack [26, 28]. Since the operation targeted by PIM is GEMV, the driver creates and delivers commands according to it, which is the same as existing PIMs. As the matrix placement is changed from row-major to column-major, AESPA has different methods of tiling matrices and generating command sequences. Fortunately, the shape and size of tiled matrices are similar and the number of generated commands is also equal or lower than previous schemes. Thus, the computational complexity and amount of command generation are at the same level compared to that of the previous PIM software.

6 METHODOLOGY

To assess the performance of the near-bank PIM architecture, we use microbenchmarks, which are level-1 (vector-vector) and level-2 (matrix-vector) BLAS operations. Except for GEMV that is a common operation of near-bank PIMs, other microbenchmarks are supported by PIM-HBM only, so used for evaluating PIM-HBM. We use GEMV microbenchmark and neural network applications [8,

Table 2: List of Evaluated Microbenchmarks and NN Applications

Microbenchmark	Description	Parameter
MUL	element-wise multiply	8 MB
ADD	element-wise add	8 MB
AXPY	element-wise multiply and add	16 MB
GEMV	4K × 2K matrix and vector	16 MB
Application	**Target Layer**	**Parameter**
AlexNet	5 CONV 3 FC	122 MB
VGGNet	13 CONV 3 FC	280 MB
MobileNet	52 CONV 1 FC	7 MB
S2S	4 LSTM	72 MB
Bert	12 TRANS	348 MB
ViT	12 TRANS	348 MB

Table 3: System Configurations

PIM-HBM configuration	
Functional unit	16-lane multiplier / 16-lane adder
Data SRAM (RF)	256 B RF per bank
Inst SRAM (RF)	32 B RF per bank
Max. parallelism	# of banks / 2
Memory	4 HBM2 stacks
HBM2 stack configuration	
Channel	8 channels, 2 pseudo channels
Rank	1 rank per channel
Bank group	4 bank groups per channel
Bank	4 banks per bank group
Bus frequency	1 GHz (DDR 2 GHz)
Device width	128 b
tCL-tRCD-tRP	14-14-14 (cycles)
Newton configuration	
Functional unit	16-lane MAC
Data SRAM (GB)	1 KB GB per channel
Max. parallelism	# of banks
Memory	4 GDDR6 dies
GDDR6 die configuration	
Channel	1 channels, 2 pseudo channels
Rank	1 rank per channel
Bank group	4 bank groups per channel
Bank	4 banks per bank group
Bus frequency	1.5 GHz (DDR 3 GHz)
Device width	16 b
tCL-tRCD-tRP	24-24-24 (cycles)

9, 13, 24, 37, 40] for evaluating both PIM-HBM and Newton architecture. Matrix-vector multiplication is included in neural network applications and classified as memory-bound operations. Many of the selected neural network operations include such matrix-vector operations, which occupy most of the total execution time. We target convolution and fully-connected layers in CNN models, LSTM layers in S2S encoder and decoder, and transformers in Bert and ViT models. Therefore, we extracted the GEMV operations included in each neural network operation and measured hardware performance. In particular, the convolution layer is converted to general matrix-matrix multiplication (GEMM). It is compute-intensive and advantageous to be accelerated in conventional processors (CPU or GPU), but evaluated PIM models can perform GEMM by conducting several GEMVs. Thus, we evaluate all possible layers to show the effectiveness of AESPA.

We modify the cycle-level DRAMsim3 [30] simulator to evaluate the performance of PIM-HBM, and Newton adopting an all-bank scheme or AESPA. The memory configuration of PIMs follows their real products [23, 29], so we adopt HBM2 for PIM-HBM and GDDR6 for Newton. The software stack, along with the hardware, helps to offload workloads from hosts to PIM devices. Thus, we conduct software stack modeling based on the previous works [26, 28]. Before execution, we divide the data and allocate them to the physical address of the bank by using a memory allocator. In the upper level of software, we use hand-coded neural network applications and microbenchmarks written by C code. In that, level-1 (MUL, ADD, and AXPY) and level-2 (GEMV) BLAS functions are called to obtain DRAM commands that execute PIMs. Depending on the type and size of operations targeted for offloading, the software stack generates an appropriate memory command trace. The PIM kernel and memory management libraries are used to determine command order and physical address. For performance evaluation, our PIM simulator receives the command trace and performs the PIM data transfer and computation.

7 EXPERIMENTAL RESULTS

In this section, we evaluate AESPA PIMs compared to the PIMs using an all-bank execution scheme. Also, we present the effectiveness of AESPA when reducing the activation overhead and varying the memory organization that affects the parallelism.

7.1 Performance Improvement of AESPA

Figure 12 shows the overall performance improvement achieved by applying AESPA to both PIM-HBM and Newton architectures. According to our experimental results, AESPA shows an average performance improvement of 59.55%. In addition, when AESPA is applied to PIM-HBM, it shows an average performance improvement of 33.54% compared to the PIM-HBM adopting all-bank execution. Moreover, when applied to Newton, it shows an average performance improvement of 85.56%. Overall, the Newton-based AESPA shows higher performance improvement than the PIM-HBM-based AESPA. This is because the proportion of activation time in the total execution time is higher in Newton due to the

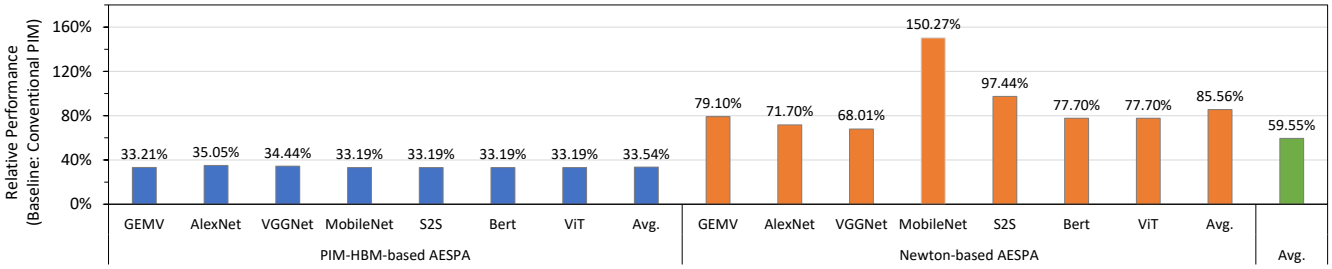

Figure 12: Relative performance improvement of AESPA

configuration of the existing architecture, which is explained in Figure 4.

Interestingly, the PIM-HBM-based AESPA shows similar performance improvements in all benchmarks. This uniformity stems from that the shape of the matrix tile remains unchanged when PIM-HBM replaces the all-bank scheme with AESPA. Thus, the performance difference of applications is equal to that of computing the single tile. On the other hand, the performance improvement of the Newton-based architecture varies depending on the benchmark application. The shape of the matrix tile is different between the all-bank scheme and AESPA. When AESPA is applied to Newton, the matrix tile features an increased number of rows. This makes AESPA particularly effective for handling tall and skinny matrices. Notably, MobileNet has tall and skinny matrices that are too small to be divided into several tiles, thereby achieving the highest performance improvement by 85.59%.

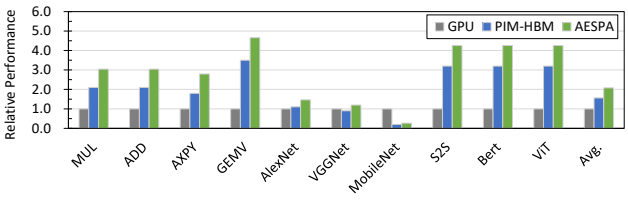

Figure 13: Relative performance compared to GPU

We also compare the performance of AESPA PIM and GPU, evaluating them across neural networks and various vector operations. Since the evaluated element-wise vector operations are not capable in the Netwon architecture. we only adopt AESPA in the PIM-HBM architecture. In the experiment, we profile GPU, RTX3090, and scale the performance of GPU by assuming that GPU has a high memory bandwidth as much as PIM-HBM. Figure 13 shows the performance of AESPA PIM and PIM-HBM when compared to that of GPU. AESPA PIM and PIM-HBM achieve average 1.7 × and 2.3 × performance improvements, respectively. The element-wise vector operations (MUL, ADD, and AXPY) are accelerated in PIM designs, which are 2.7-4.6 × higher performance than that of GPU. In the case of workloads that consist of GEMV mainly (fully-connected layers of S2S, Bert, and ViT), PIM designs outperform GPU by 3.2-4.7 ×. On the other hand, workloads including GEMM (convolution layers of AlexNet, VGGNet, and MobileNet) show low performance in PIM designs as 0.2-1.4 × of GPU performance.

7.2 Impact of Activation Overhead

AESPA is designed to reduce the overhead resulting from activations in existing near-bank PIM architectures. However, the prior studies [12, 29] have tackled activation overhead through the circuit-level approach by supplying additional voltage. They do not provide timing parameters that could be reduced by changing the DRAM circuit. Thus, we conduct an experiment to evaluate AESPA PIMs and previous near-bank PIMs by varying activation overhead instead. Figure 14 shows the relative execution delay of the AESPA PIM with 100% overhead and PIM architectures with 100%, 75%, 50%, 25%, and 0% overhead. The 100% overhead is measured in the original execution, which includes the tFAW overhead and performs per-bank activations. On the other hand, the 0% overhead assumes that the PIM execution has no tFAW overhead and performs the all-bank activation in a single per-bank activation delay. It shows that both PIM-HBM and Newton exhibit a decrease in execution time proportional to the decrease in activation overhead. Additionally, the ratio of operation cycles per activation cycle (OP/ACT in the figure) also gradually increases. However, we find that the execution time does not decrease as much as AESPA even when the PIMs have no additional delays caused by activations (0%). In both architectures, PIM-HBM and Newton, the single activation delay exists in the 0% overhead whereas the AESPA PIMs also hide the single activation delay by overlapping to long-data commands. Through this experiment, we observe that AESPA has a similar or higher performance impact compared to the circuit-level approach.

7.3 Impact of the Memory Configuration

When changing memory configuration, AESPA PIM operates with the same mechanisms, but parallelism and overhead might vary. By increasing the number of banks, the performance of baseline architectures is improved by exploiting the bank-level parallelism but tFAW occurs more frequently. On the other hand, when increasing the size of the row, the number of columns is increased. It is advantageous to exploit the parallelism of AESPA but the tFAW parameter becomes much longer. In our experiment, a DRAM bank utilizing a 2048 byte row required a tFAW of 25.2 ns whereas a DRAM bank utilizing a 128 byte row required a tFAW of 8.5 ns.

To assess performance across different bank configurations, we conducted design space exploration by varying the number of banks and the size of the row. Figure 15 presents a heatmap of the performance improvement achieved when applying AESPA to PIM-HBM and Newton. The base memory organization described in Table 3

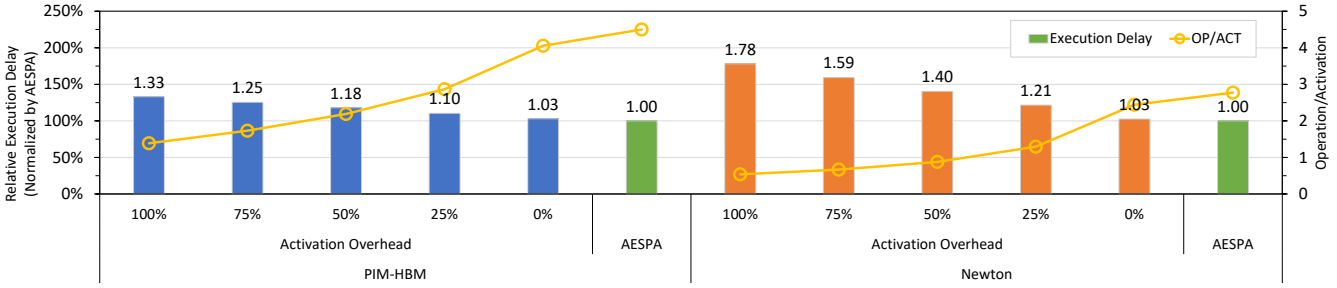

Figure 14: Total execution time with activation overhead reduction

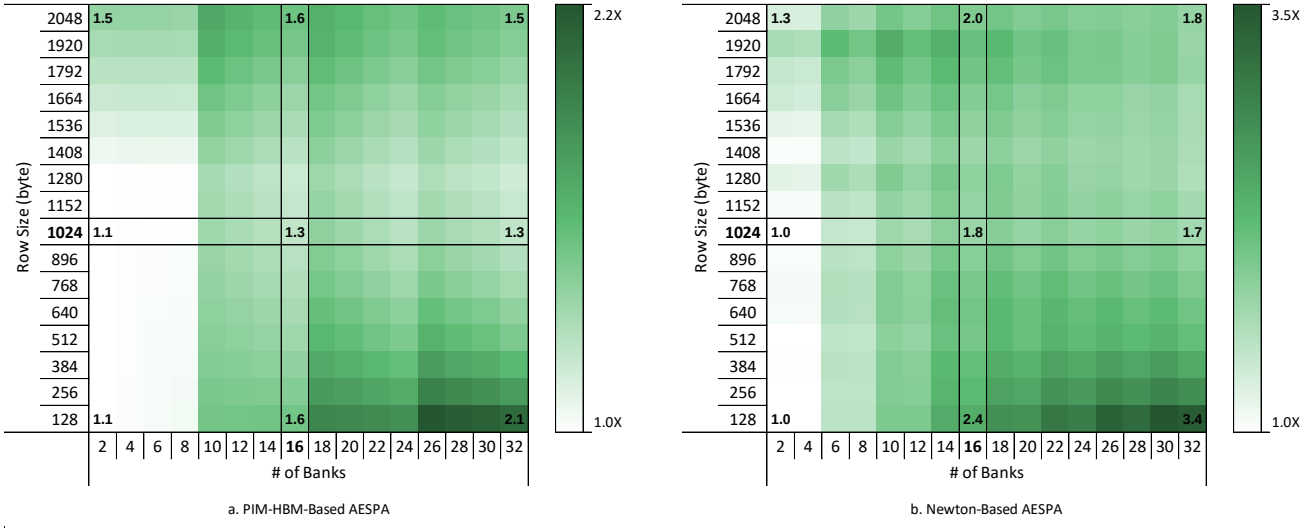

Figure 15: Performance improvement when varying the size of row and number of banks

is placed in the center of the figure and the darker green indicates higher performance improvement. In the base memory organization, AESPA already delivers performance improvements of 1.3× and 1.8× when compared to PIM-HBM and Newton, respectively. Interestingly, AESPA always outperforms baseline execution (i.e., performance improvement over 1.0×). Adopting AESPA is especially better for memory organization with many banks and small rows (32 banks with 128-byte). This is because AESPA mitigates the frequent occurrence of tFAW and activation delays while preserving parallelism, resulting in substantial performance gains.

When increasing the number of banks (32 banks with 1024-byte row), the performance improvements are not or few changed as 1.3× and 1.4×. In the opposite case, when decreasing the number of banks (2 banks with 1024-byte row), the performance improvements are reduced by 1.1× and 1.0×. It means that reducing the number of banks avoids frequent tFAW delays. While this is advantageous for all-bank execution schemes, AESPA still achieves the same or larger performance compared to baseline execution. Meanwhile, when increasing or decreasing the size of DRAM rows (16 banks with 2048-byte row or 128-byte row), both organizations further increase the performance improvement of AESPA. The PIM-HBM-based AESPA PIM improves 1.6× in both organizations and the Newton-based AESPA PIM achieves 2.0× and 2.4× improvements in large-row and small-row organizations. This is because increasing row size incurs high tFAW overhead and reducing row size reduces the ratio of activation delay to execution delay (OP/ACT).

7.4 Area and Power Overhead

To replace the execution scheme, we changed the hardware of the PIM adopted to AEPA. By doing so, additional hardware components are used such as data bus and SRAM structures. Thus, we evaluate the area and power overhead of the bank by assuming PIM architectures fabricated using 29 nm technology. We utilize Synopsis Design Compiler [41] for the processing unit, CACTI [32] for SRAM structures, DRAMspec [43] for the area overhead of DRAM structures, and DRAMsim3 for power overheads. The power density is estimated by the dynamically consumed power of the processing unit and bank.

The results of area and power density per bank are presented in Table 4. In both PIM-HBM and Newton, AESPA slightly increases the area of the processing unit. It is due to the accumulation of accumulators in bank groups. This area overhead was mainly caused by additional I/O receiving the operation results for each bank and the DRAM row table, which can identify the source of data. For both

Table 4: Area and Power Overhead per Bank

Model / Scheme	PIM Component	Area (mm^2)	Power density (W/mm^2)
PIM-HBM / Baseline	PU	0.094	41.3
	Bank	0.687	63.8
PIM-HBM / AESPA	PU	0.105	41.1
	Bank	0.687	68.8
Newton / Baseline	PU	0.102	41.1
	Bank	0.687	63.8
Newton / AESPA	PU	0.110	41.2
	Bank	0.687	68.2

models, when compared to the baseline, applying AESPA results in a 10% larger area for the processing unit. Regarding processing unit power density, there was no significant difference between the baseline and AESPA in both PIM-HBM and Newton. However, the power density is slightly higher when AESPA is applied, as the same amount of DRAM operations are performed in a shorter time.

8 RELATED WORK

Near-bank PIM: Recently, various practical PIMs have been taped out and attracted attention [6, 23, 29]. In addition, efforts are made to discover use cases for these PIMs [19, 21, 27]. Among them, near-bank PIM can utilize high bandwidth of many banks, so it is being utilized in various ways such as building a PIM system [6]. Samsung Aquabolt-XL [23] and SK Hynix AiM-GDDR6 [29] are ASIC-style near-bank PIMs and mainly used to accelerate matrix multiplication and simple operations in NN applications. On the other hand, UPMEM DPU [6] supports general processing based on 8-bit integer computations, so there have been studies using it for sparse matrix operations for the graph processing [10]. Also, to utilize near-bank PIMs in general-purpose computing, Devic et al. present an analysis of workloads and provide framework design [7], which decides the PIM or host processor process workload.

All-bank Execution: Most near-bank PIM research assumes that all banks operate simultaneously [5, 36, 44]. Some studies assume that processors within each bank perform their own processes without involving the host processor. However, this needs to be managed by the host processor in terms of channel bus congestion and memory power management. Therefore, commercial PIMs can support all-bank commands and improve existing memory constraints for that [23, 29]. Since this approach requires modifications to the actual memory circuit, research has been conducted to enable all-bank PIMs while maintaining the standard JEDEC DRAM interface [33]. PIMs are designed to place the required data for each bank in the PIM engine during the memory phase and perform simultaneous operations in the compute phase.

Matrix multiplication modification: In GPU and NN accelerator research, there are often studies that change the operation order of matrix multiplication [34, 42, 45]. In particular, there have been several techniques to change the operation to an outer product-based approach instead of the inner product-based approach that was mainly used in sparse matrix multiplication. This utilizes the characteristics of outer product-based matrix multiplication, which allows for the efficient reuse of input vectors and matrices [45]. It has mainly been used to resolve operation irregularities in compressed format operations.

9 CONCLUSION

In this paper, we introduce AESPA, a novel PIM architecture that leverages bank-level parallelism. It enables all banks to function asynchronously, avoiding the need for all-bank activation. AESPA employs a distinct matrix operation mechanism and data placement when compared to conventional near-bank PIMs. These techniques allow each bank to operate independently with different commands. AESPA can start operations asynchronously and operate all banks simultaneously. As a result, our approach effectively hides the delays caused by activations while still preserving parallelism.

ACKNOWLEDGMENTS

This research was supported in part by the Ministry of Trade, Industry, and Energy (MOTIE) under Grant 1415181094 and in part by the Korea Semiconductor Research Consortium (KSRC) Support Program for the Development of the Future Semiconductor Device under Grant 20019956. This work was also supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00441, Memory-Centric Architecture Using the Reconfigurable PIM Devices). Won Woo Ro is the corresponding author.

REFERENCES

[1] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. 2015. PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture* (Portland, Oregon) *(ISCA '15)*. Association for Computing Machinery, New York, NY, USA, 336–348. https://doi.org/10.1145/2749469.2750385

[2] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. 2002. An updated set of basic linear algebra subprograms (BLAS). *ACM Trans. Math. Software* 28, 2 (2002), 135–151.

[3] Kevin Kai-Wei Chang, Donghyuk Lee, Zeshan Chishti, Alaa R Alameldeen, Chris Wilkerson, Yoongu Kim, and Onur Mutlu. 2014. Improving DRAM performance by parallelizing refreshes with accesses. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 356–367.

[4] Benjamin Y. Cho, Yongkee Kwon, Sangkug Lym, and Mattan Erez. 2020. Near Data Acceleration with Concurrent Host Access. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 818–831. https://doi.org/10.1109/ISCA45697.2020.00072

[5] Seunghwan Cho, Haerang Choi, Eunhyeok Park, Hyunsung Shin, and Sungjoo Yoo. 2020. McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge. *IEEE Access* 8 (2020), 135223–135243. https://doi.org/10.1109/ACCESS.2020.3011265

[6] Fabrice Devaux. 2019. The true processing in memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 1–24.

[7] Alexandar Devic, Siddhartha Balakrishna Rai, Anand Sivasubramaniam, Ameen Akel, Sean Eilert, and Justin Eno. 2022. To PIM or Not for Emerging General Purpose Processing in DDR Memory Systems. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 231–244. https://doi.org/10.1145/3470496.3527431

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[10] Christina Giannoula, Ivan Fernandez, Juan Gómez-Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Towards efficient sparse matrix

vector multiplication on real processing-in-memory systems. *arXiv preprint arXiv:2201.05072* (2022).

[11] Peng Gu, Xinfeng Xie, Yufei Ding, Guoyang Chen, Weifeng Zhang, Dimin Niu, and Yuan Xie. 2020. iPIM: Programmable in-memory image processing accelerator using near-bank architecture. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 804–817.

[12] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and TN Vijaykumar. 2020. Newton: A DRAM-maker's accelerator-in-memory (AiM) architecture for machine learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 372–385.

[13] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).

[14] Intel. 2023. Accelerate Fast Math with Intel® oneAPI Math Kernel Library.

[15] Spec JEDEC. 2013. High bandwidth memory (hbm) dram. *JESD235* (2013), 0018–9340.

[16] JEDEC JESD250. 2017. Graphics double data rate 6 (GDDR6) SGRAM standard. *JEDEC Solid State Technology Association* (2017).

[17] Hongju Kal, Cheolhwan Kim, Minjae Kim, and Won Woo Ro. 2023. Context Swap: Multi-PIM System Preventing Remote Memory Access for Large Embedding Model Acceleration. In *2023 IEEE 5th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 1–5.

[18] Hongju Kal, Seokmin Lee, Gun Ko, and Won Woo Ro. 2021. Space: locality-aware processing in heterogeneous memory for personalized recommendations. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 679–691.

[19] Shin-haeng Kang, Byeongho Kim, Sukhan Lee, and Kyomin Sohn. 2022. An Architecture of Sparse Length Sum Accelerator in AxDIMM. In *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 1–4.

[20] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. 2020. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 790–803.

[21] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. 2022. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro* 42, 1 (2022), 116–127. https://doi.org/10.1109/MM.2021.3097700

[22] Jin Hyun Kim, Shin-Haeng Kang, Sukhan Lee, Hyeonsu Kim, Yuhwan Ro, Seungwon Lee, David Wang, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jeonghyeon Cho, Kyomin Sohn, and Nam Sung Kim. 2022. Aquabolt-XL HBM2-PIM, LPDDR5-PIM With In-Memory Processing, and AXDIMM With Acceleration Buffer. *IEEE Micro* 42, 3 (2022), 20–30. https://doi.org/10.1109/MM.2022.3164651

[23] Jin Hyun Kim, Shin-haeng Kang, Sukhan Lee, Hyeonsu Kim, Woongjae Song, Yuhwan Ro, Seungwon Lee, David Wang, Hyunsung Shin, Bengseng Phuah, et al. 2021. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 1–26.

[24] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2017. Imagenet classification with deep convolutional neural networks. *Commun. ACM* 60, 6 (2017), 84–90.

[25] Youngeun Kwon, Yunjae Lee, and Minsoo Rhu. 2019. Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 740–753.

[26] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, et al. 2022. System architecture and software stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 1–25.

[27] Donghun Lee, Jinin So, Minseon Ahn, Jong-Geon Lee, Jungmin Kim, Jeonghyeon Cho, Rebholz Oliver, Vishnu Charan Thummala, Ravi shankar JV, Sachin Suresh Upadhya, et al. 2022. Improving in-memory database operations with acceleration DIMM (AxDIMM). In *Data Management on New Hardware*. 1–9.

[28] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, et al. 2021. Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 43–56.

[29] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, et al. 2022. A 1ynm 1.25 v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. IEEE, 1–3.

[30] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.

[31] Daichi Mukunoki, Toshiyuki Imamura, and Daisuke Takahashi. 2015. Fast implementation of general matrix-vector multiplication (GEMV) on Kepler GPUs. In *2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. IEEE, 642–650.

[32] Naveen Muralimanohar, Rajeev Balasubramonian, and Norm Jouppi. 2007. Optimizing NUCA Organizations and Wiring Alternatives for Large Caches with CACTI 6.0. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*. 3–14. https://doi.org/10.1109/MICRO.2007.33

[33] Yoonah Paik, Chang Hyun Kim, Won Jun Lee, and Seon Wook Kim. 2022. Achieving the Performance of All-Bank In-DRAM PIM With Standard Memory Interface: Memory-Computation Decoupling. *IEEE Access* 10 (2022), 93256–93272. https://doi.org/10.1109/ACCESS.2022.3203051

[34] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. 2018. OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 724–736. https://doi.org/10.1109/HPCA.2018.00067

[35] Albert Reuther, Peter Michaleas, Michael Jones, Vijay Gadepally, Siddharth Samsi, and Jeremy Kepner. 2022. AI and ML Accelerator Survey and Trends. In *2022 IEEE High Performance Extreme Computing Conference (HPEC)*. 1–10. https://doi.org/10.1109/HPEC55821.2022.9926331

[36] Hyunsung Shin, Dongyoung Kim, Eunhyeok Park, Sungho Park, Yongsik Park, and Sungjoo Yoo. 2018. McDRAM: Low Latency and Energy-Efficient Matrix Computations in DRAM. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 11 (2018), 2613–2622. https://doi.org/10.1109/TCAD.2018.2857044

[37] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).

[38] JEDEC Standard. 2012. JEDEC Standard: DDR4 SDRAM. *JESD79-4, Sep* (2012).

[39] JEDEC Standard. 2013. High bandwidth memory (hbm) dram. *JESD235* (2013).

[40] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. *Advances in neural information processing systems* 27 (2014).

[41] Synopsys. 2023. Synopsys Design Compiler. http://www.synopsys.com.

[42] Yang Wang, Chen Zhang, Zhiqiang Xie, Cong Guo, Yunxin Liu, and Jingwen Leng. 2021. Dual-side Sparse Tensor Core. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 1083–1095. https://doi.org/10.1109/ISCA52012.2021.00088

[43] Christian Weis, Abdul Mutaal, Omar Naji, Matthias Jung, Andreas Hansson, and Norbert Wehn. 2017. Dramspec: A high-level dram timing, power and area exploration tool. *International Journal of Parallel Programming* 45, 6 (2017), 1566–1591.

[44] Xinfeng Xie, Zheng Liang, Peng Gu, Abanti Basak, Lei Deng, Ling Liang, Xing Hu, and Yuan Xie. 2021. SpaceA: Sparse Matrix Vector Multiplication on Processing-in-Memory Accelerator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 570–583. https://doi.org/10.1109/HPCA51647.2021.00055

[45] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. 2020. SpArch: Efficient Architecture for Sparse Matrix Multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–274. https://doi.org/10.1109/HPCA47549.2020.00030